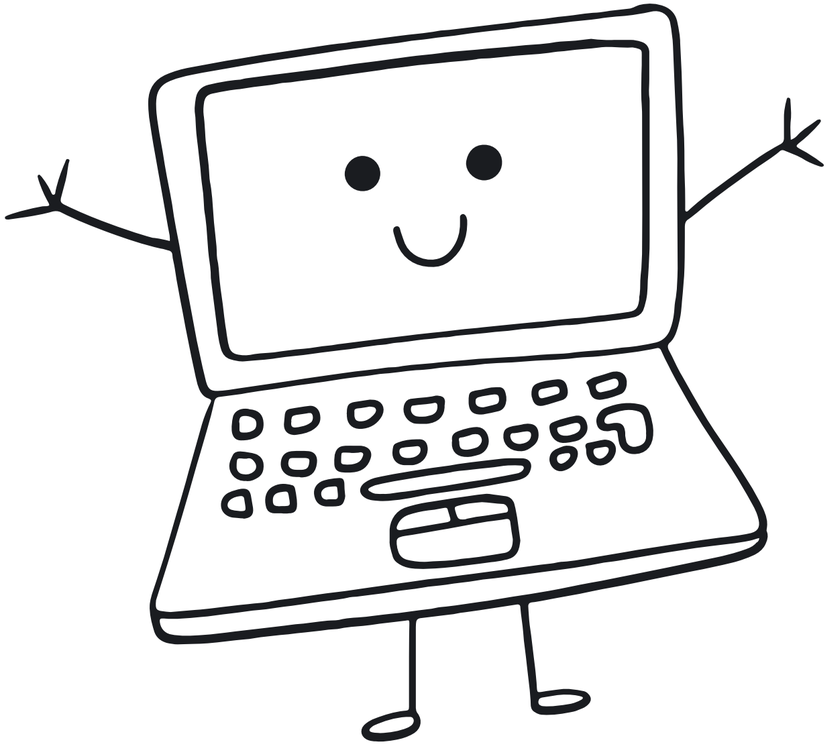
RETRIEVAL AUGMENTED GENERATION IN .NET

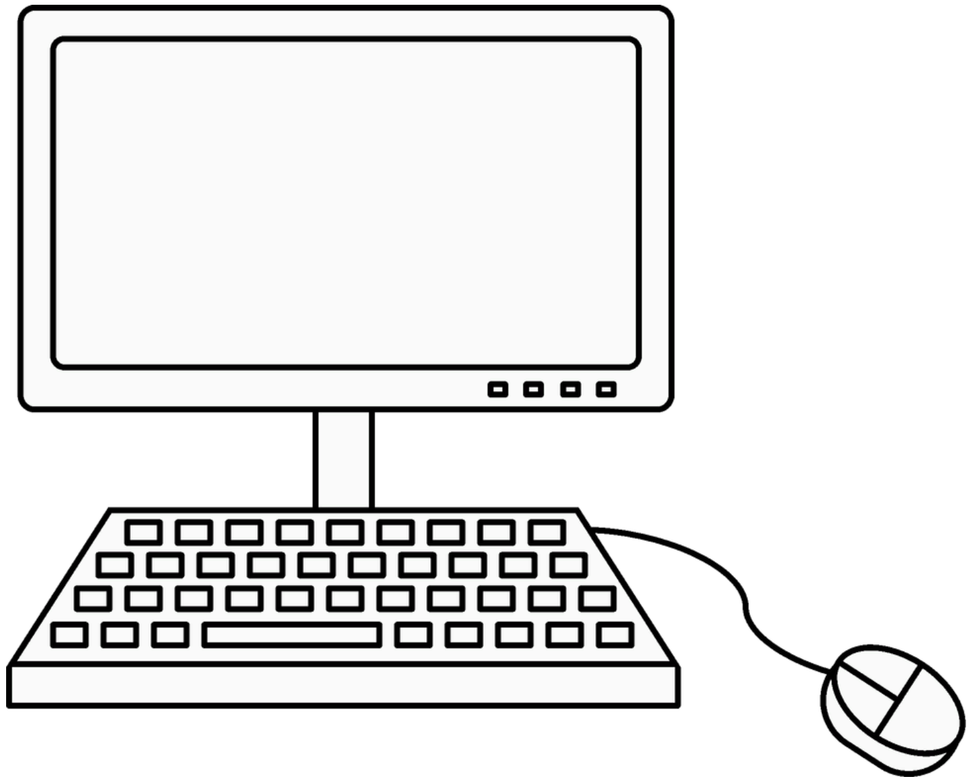
(WITHOUT VECTOR DATABASE)

TUAN.MN@KMS-TECHNOLOGY.COM

WELCOME

- **Project**: RAG Without Vector Database
- **Workspace**: RAGSolution
- **Core idea:** Retrieval-Augmented Generation using lexical ranking (no embeddings)
- **Stack**: .NET 9 + Sementic Kernal + Vertical Slice Architecture (REPR Pattern + FastEndpoints) + SQL Server + OpenAI + React/Vite





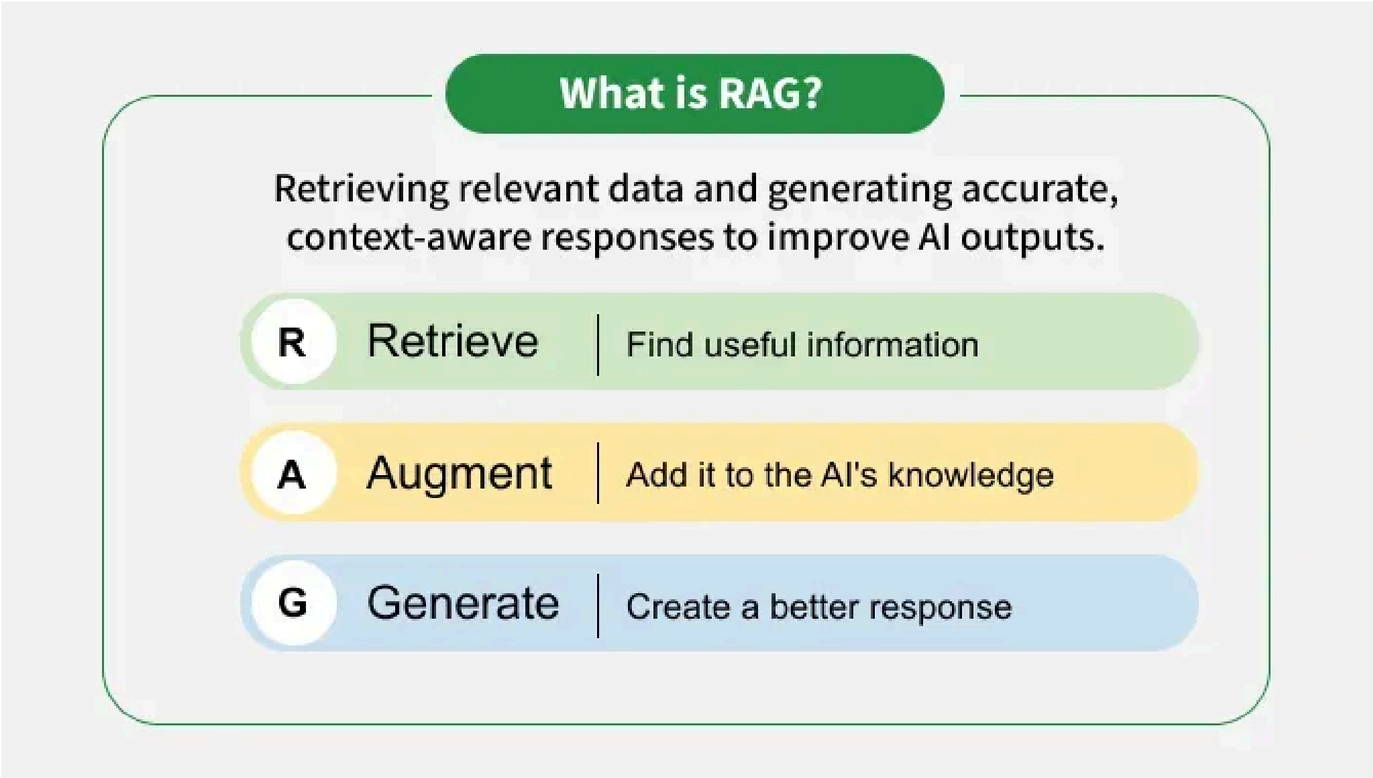
OUTLINE

- RAG Definition
- Which one of LLMs in the Project
- Utilisation of Open AI Platform
- Project Demo
- Available Model Options
- Project Functionality Workflow
- High-Level and Low-Level Aspects
- Strengths, Trade-Offs, and Next Steps

RAG Definition

(Retrieval-Augmented Generation) is an AI architecture that combines information retrieval with text generation to produce more accurate, grounded, and up-to-date responses.

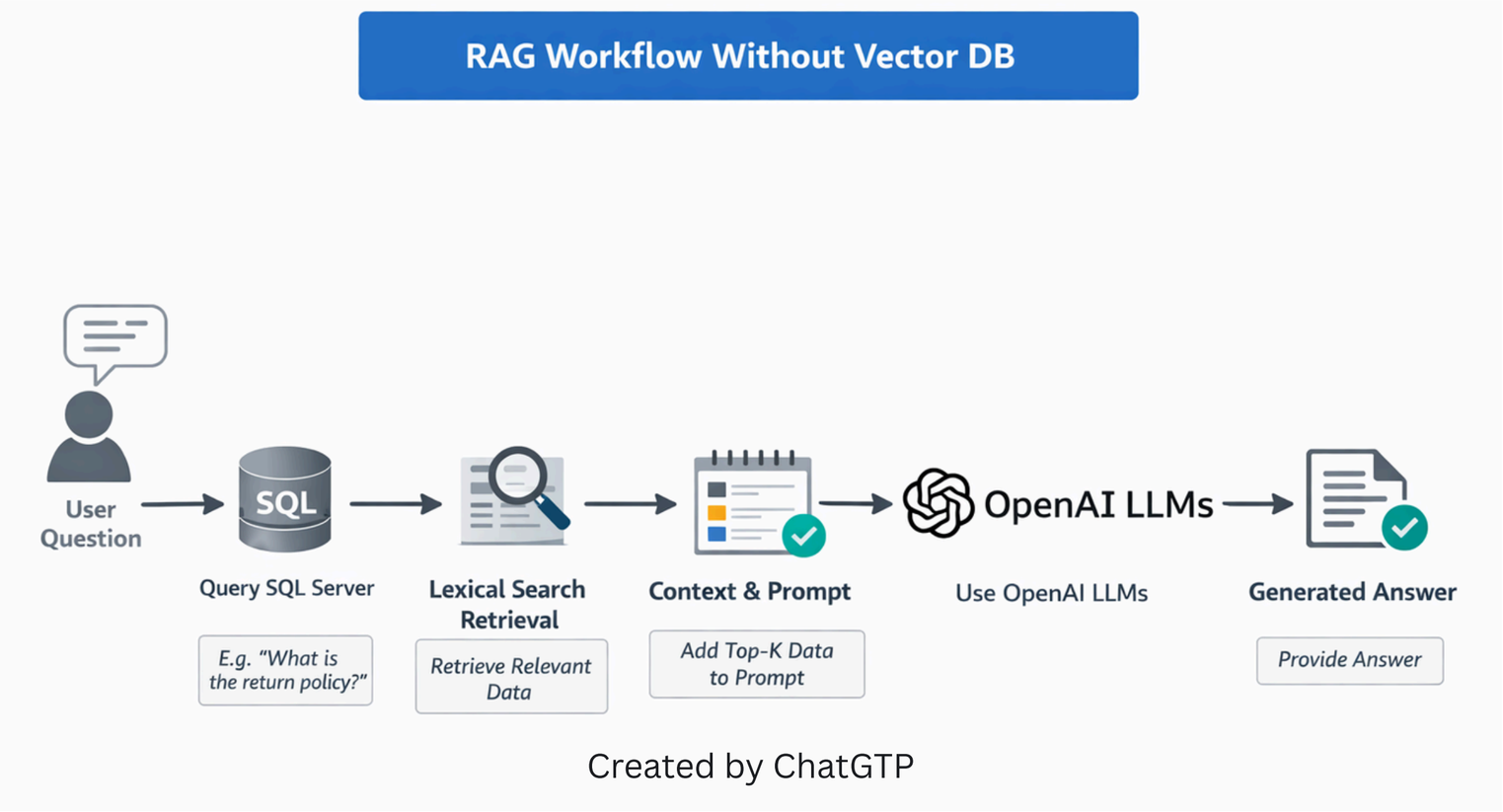
RAG = Retrieve relevant data → then Generate an answer using that data



https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/

RAG Definition

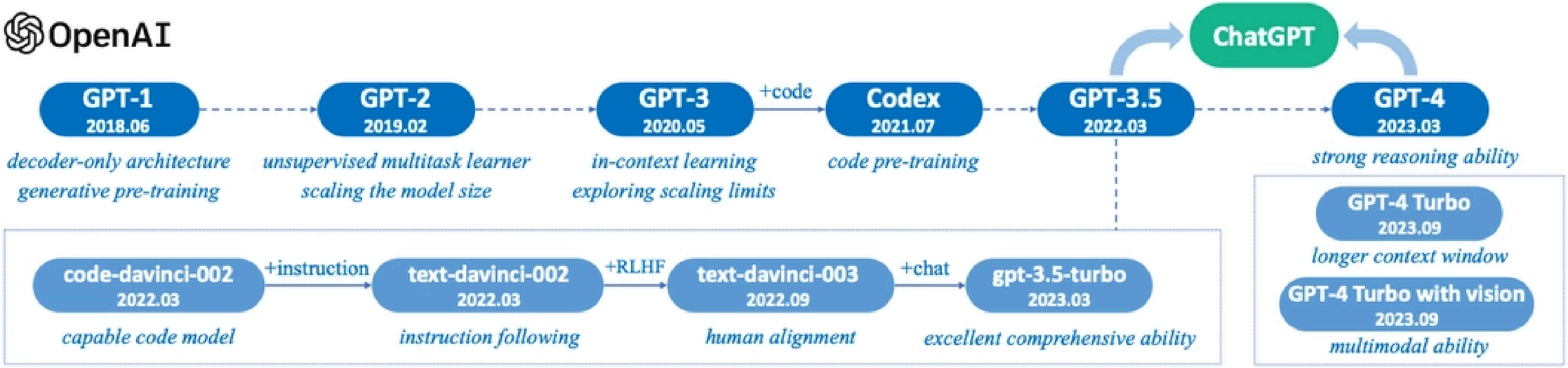
1. User question
2. Query SQL Server
3. Lexical Search Retrieval
4. Context & Prompt
5. Use OpenAI LLMs
6. Generated answer



Created by ChatGTP

Which one of LLMs in the Project

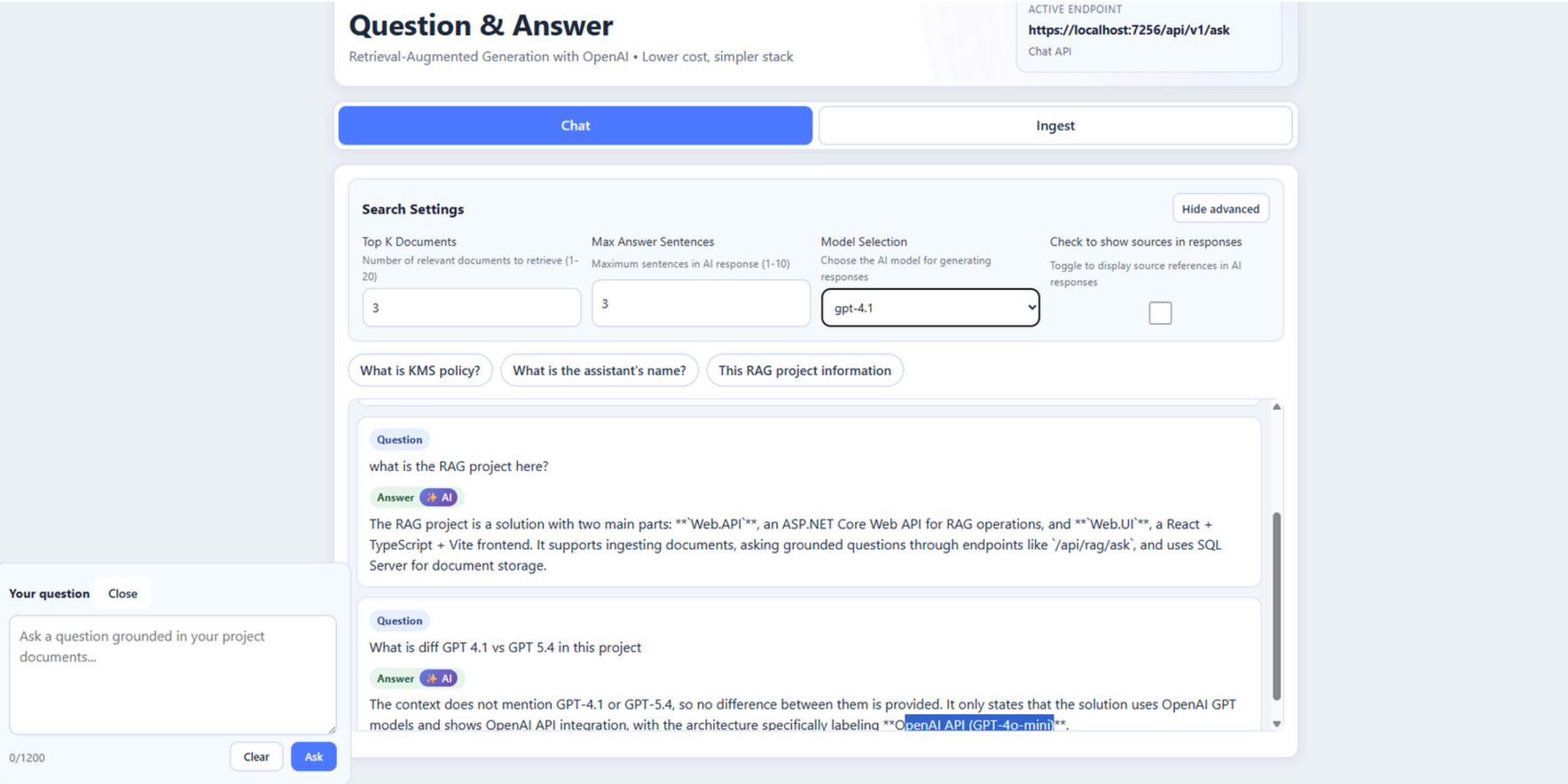
Large Language Models (LLMs) are AI systems designed to understand, process, and generate human-like text, code, and other content by identifying patterns in massive datasets



https://klu.ai/glossary/large-language-model

Which one of LLMs in the Project

- gpt-4.1
- gpt-5.4
- gpt-3.5 Turbo



Demo Project

Which one of LLMs in the Project

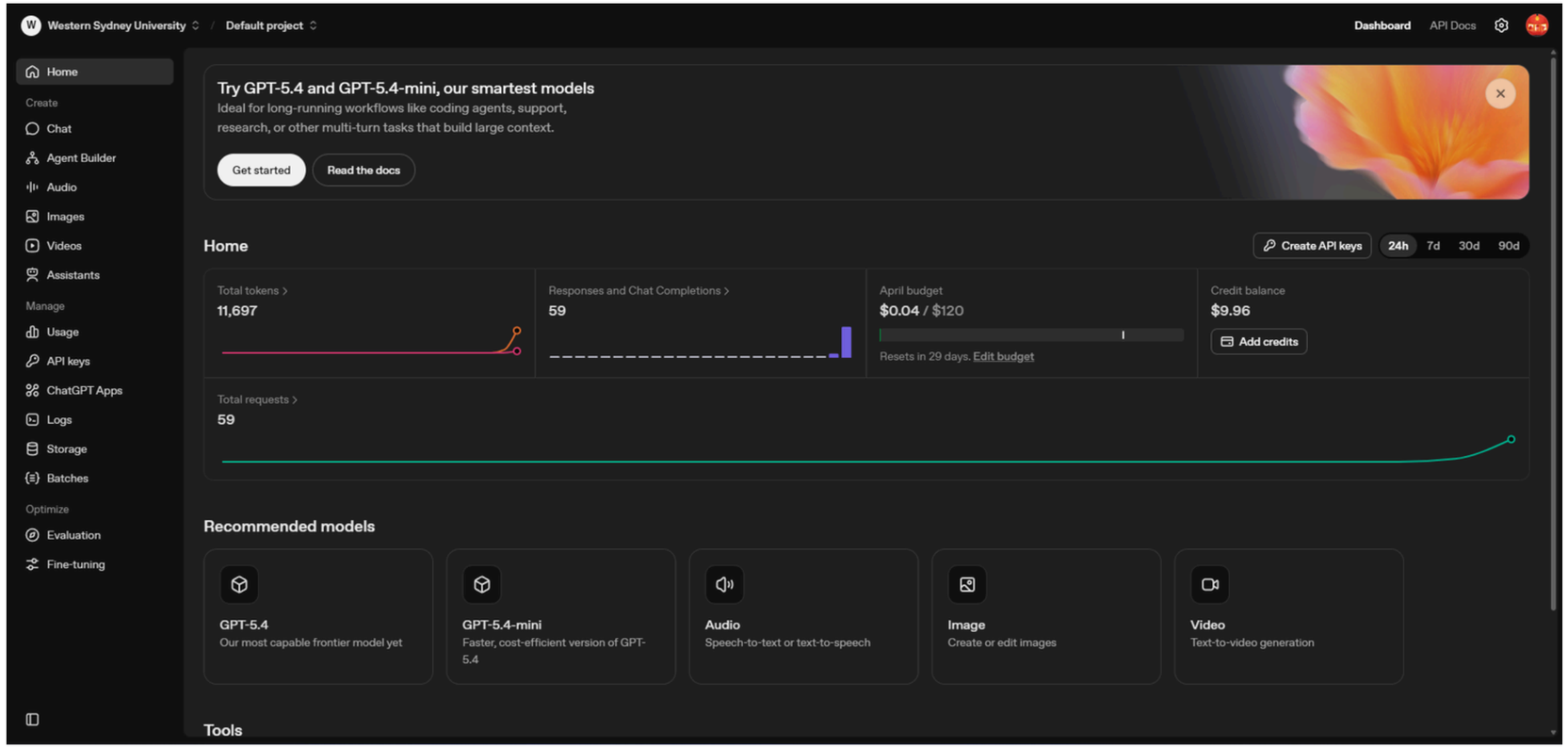
Model Overview

Feature	GPT-3.5 Turbo	GPT-4.1 Nano
Input Context Window The number of tokens supported by the input context window.	**16.4K** tokens	**1M** tokens
Maximum Output Tokens The number of tokens that can be generated by the model in a single request.	**4,096** tokens	**32K** tokens
Open Source Whether the model's code is available for public use.	**No**	**No**
Release Date When the model was first released.	**January 25, 2024** 2 years ago	**April 14, 2025** 11 months ago
Knowledge Cut-off Date When the model's knowledge was last updated.	**September 2021**	**June 2024**
API Providers The providers that offer this model. (This is not an exhaustive list.)	OpenAI, Azure OpenAI Service	OpenAI API
Supported Modalities The types of inputs the model can process.		

https://docsbot.ai/models/compare/gpt-3-5-turbo/gpt-4-1-nano

UTILISATION OF OPEN AI PLATFORM

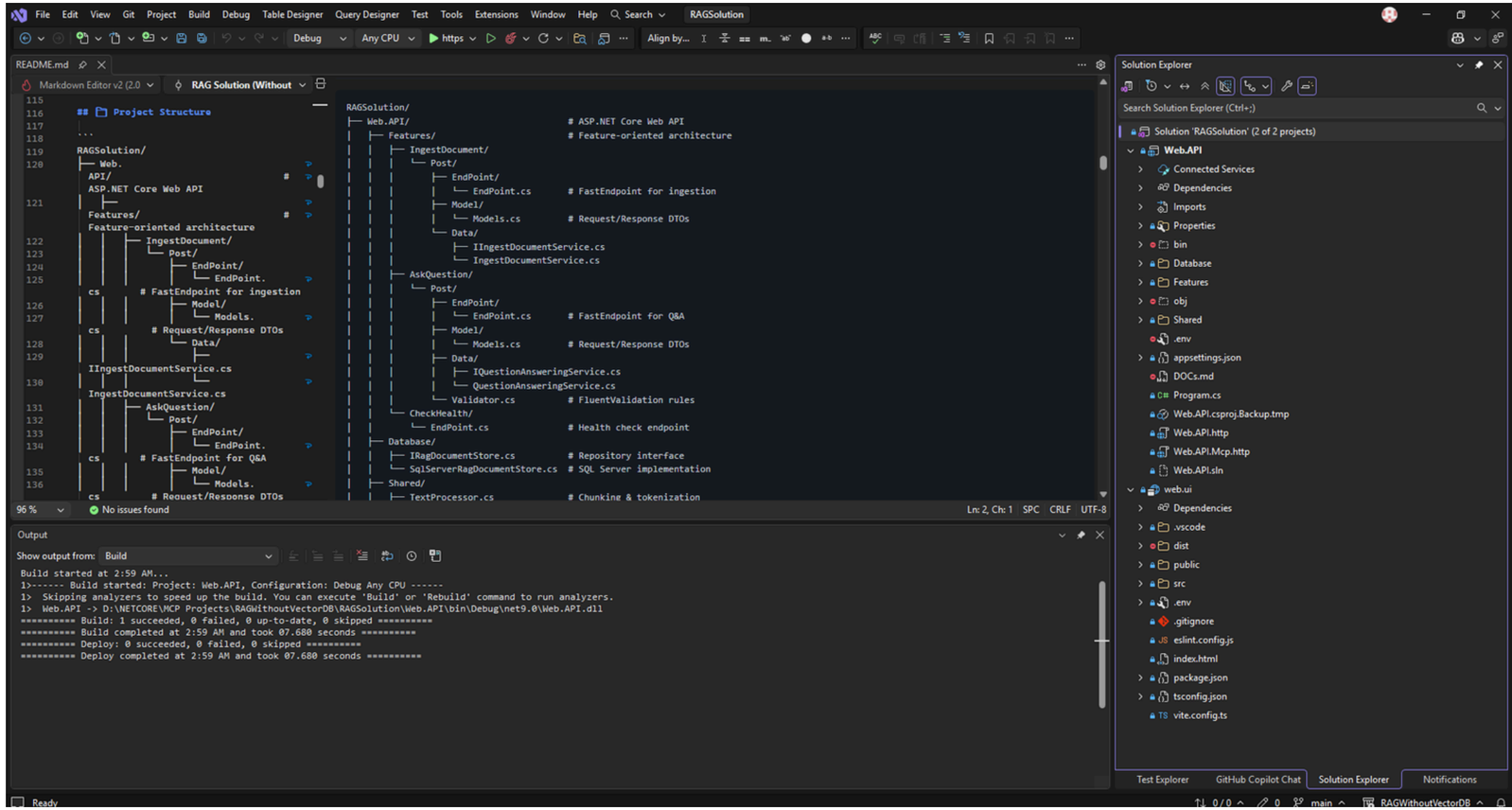
Overveiew of OpenAI platform and its services



PROJECT DEMO

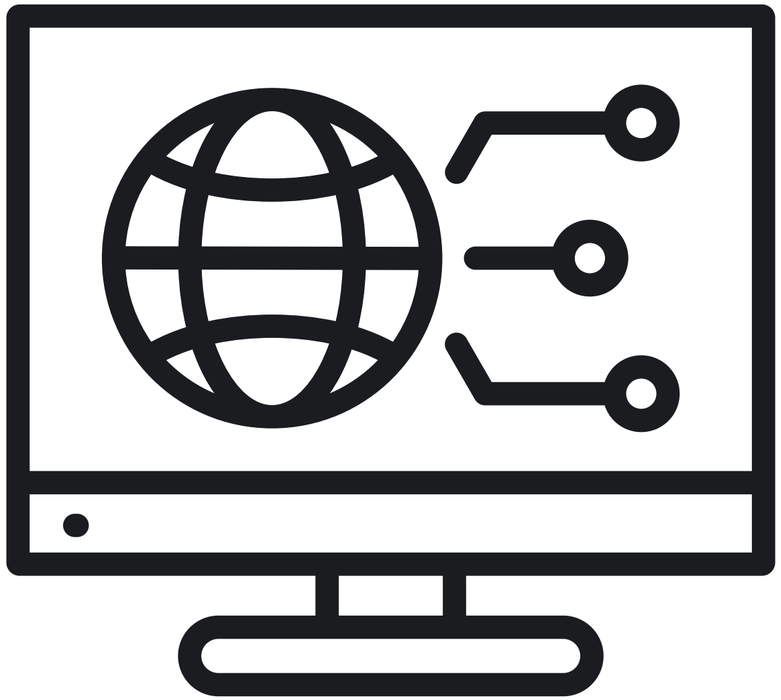
- Ingest feature
- Ask feature

```
├── Web.API/                          # ASP.NET Core Web API
│   ├── Features/                     # Feature-oriented architecture
│   │   ├── IngestDocument/
│   │   │   └── Post/
│   │   │       ├── EndPoint/
│   │   │       │   └── EndPoint.cs       # FastEndpoint for ingestion
│   │   │       ├── Model/
│   │   │       │   └── Models.cs         # Request/Response DTOs
│   │   │       └── Data/
│   │   │           ├── IIngestDocumentService.cs
│   │   │           └── IngestDocumentService.cs
│   │   ├── AskQuestion/
│   │   │   └── Post/
│   │   │       ├── EndPoint/
│   │   │       │   └── EndPoint.cs       # FastEndpoint for Q&A
│   │   │       ├── Model/
│   │   │       │   └── Models.cs         # Request/Response DTOs
│   │   │       ├── Data/
│   │   │       │   ├── IQuestionAnsweringService.cs
│   │   │       │   └── QuestionAnsweringService.cs
│   │   │       └── Validator.cs          # FluentValidation rules
│   │   └── CheckHealth/
│   │       └── EndPoint.cs               # Health check endpoint
│   ├── Database/
│   │   ├── IRagDocumentStore.cs          # Repository interface
│   │   └── SqlServerRagDocumentStore.cs  # SQL Server implementation
│   ├── Shared/
│   │   ├── TextProcessor.cs              # Chunking & tokenization
│   │   ├── RelevanceScorer.cs            # Lexical scoring algorithm
│   │   └── InternalModels.cs             # Shared domain models
│   ├── Program.cs                        # Application entry point
│   ├── Web.API.csproj                    # Project file
│   ├── .env                              # Environment variables (API keys)
│   ├── appsettings.json                  # Configuration
│   └── appsettings.Development.json      # Dev configuration
│
```



AVAILABLE MODEL OPTIONS

- Customise model to generate text by OpenAI
- Implement additional AI systems like Azure Open AI or Google Gemini



FRONT-END

- React JS
- Created by Copilot
- Using feature layers and shared infrastructure layer

BACK-END

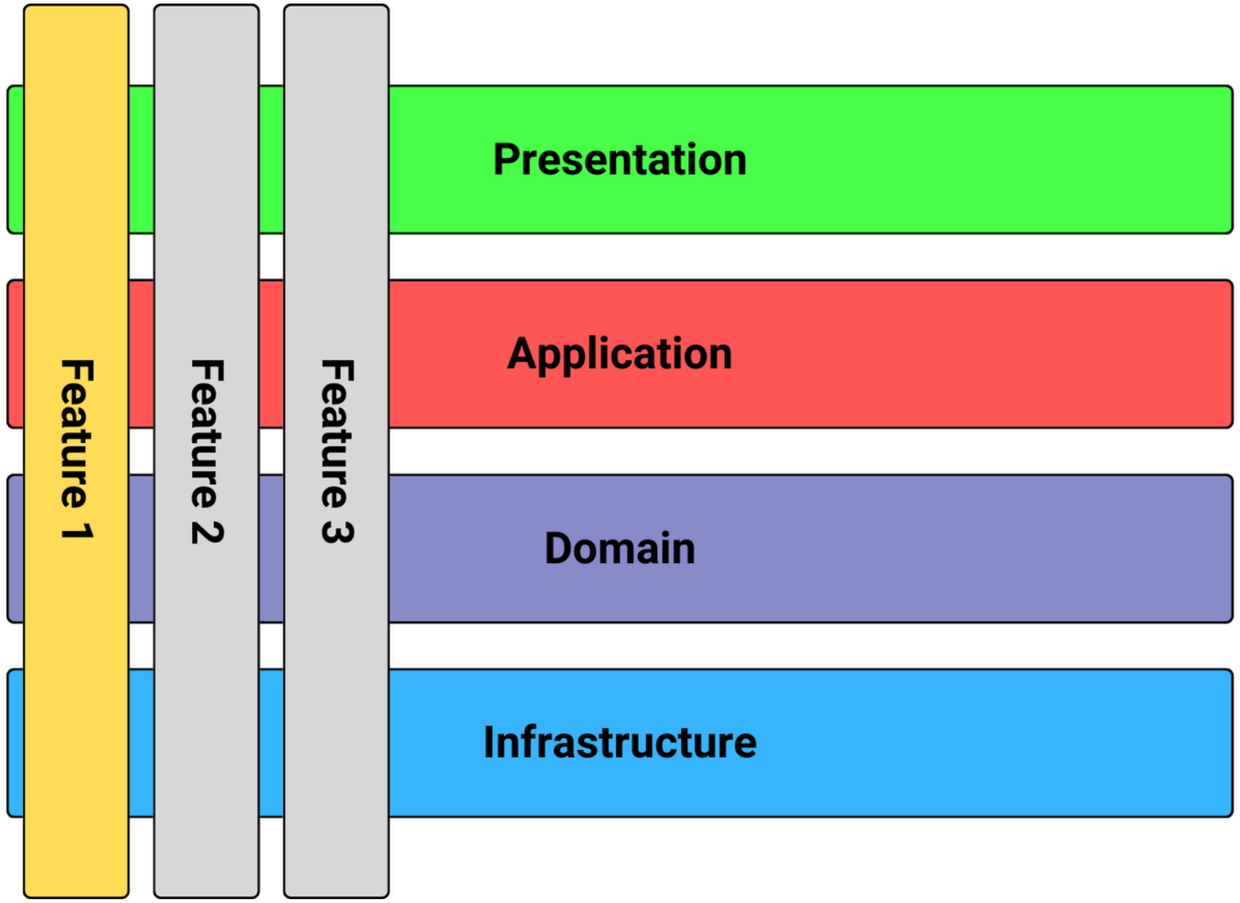
- .NET 9
- Vertical Slice Architecture
- REPR Pattern
- FastEndPoint
- (SQL Server)

PROJECT FUNCTIONALITY WORKFLOW

VERTICLE SLICE ARCHITECTURE

Vertical Slice Architecture is a software design approach that organizes code around features or use cases rather than technical layers (e.g., UI, Business Logic, Data Access).

(Read more)



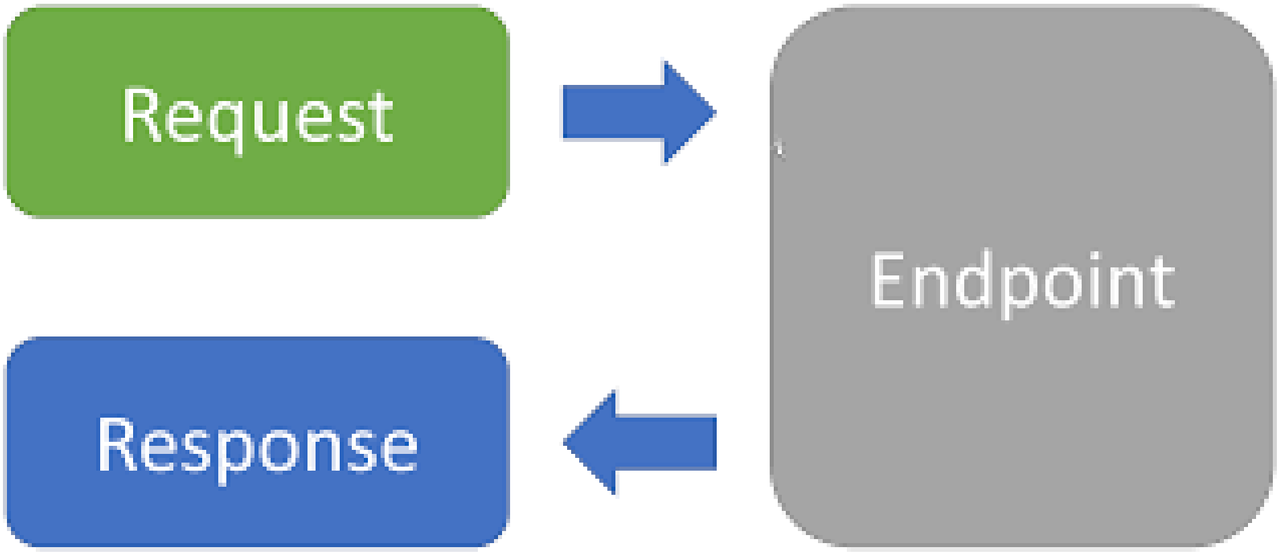
https://www.milanjovanovic.tech/blog/vertical-slice-architecture

PROJECT FUNCTIONALITY WORKFLOW

REPR PATTERN

- Endpoint
- Request
- Response

Read more



HIGH-LEVEL ASPECTS (WHAT THE SYSTEM DOES)

- Delivers grounded Q&A over ingested documents without a vector database.
- Uses a two-tier application architecture:
- React frontend for user interaction.
- ASP.NET Core API for ingestion, retrieval, ranking, and answer generation.
- Persists documents in SQL Server and augments responses with source snippets.
- Exposes versioned API contracts (v1 and v2) for ask, ingest, and health operations.
- Supports operational observability through health endpoints and v2 runtime diagnostics.

LOW-LEVEL ASPECTS (HOW IT IS IMPLEMENTED)

Routing/versioning:
- FastEndpoints route prefix `api` with prepended version segment `v{version}`.

Ingestion internals:
- Content split by sentence regex and grouped into 3-sentence chunks.
- Tokens are normalized (alphanumeric, lowercase, short token filtering).
- SQL persistence uses `MERGE` upsert into `dbo.RagDocuments`.

Retrieval internals:
- Documents are loaded and cached in-memory in question-answer service.
- Relevance scoring uses lexical overlap + density bonus per chunk.
- Top-K selection clamps request range to 1..10.

Generation internals:
- Selected chunks are concatenated into prompt context.
- OpenAI chat completion returns final answer text.

Contract internals:
- Validators enforce required question/content fields and numeric bounds.
- v2 responses enrich v1 payloads with processing metadata.

STRENGTHS

- Cost-Effective
- Simpler Architecture (React → .NET API → SQL Query → Top-K Results → Prompt → LLM → Response)
- No Embedding Overhead

TRADE-OFFS

- Weak Semantic Understanding
- Lower Recall Quality
- Performance with Large Text Search
- LLM Dependency Increases

NEXT STEPS

- Search Tecniques Improvement
- Performance Boost
- Storage other info (Ex: chat history)
- Add caches

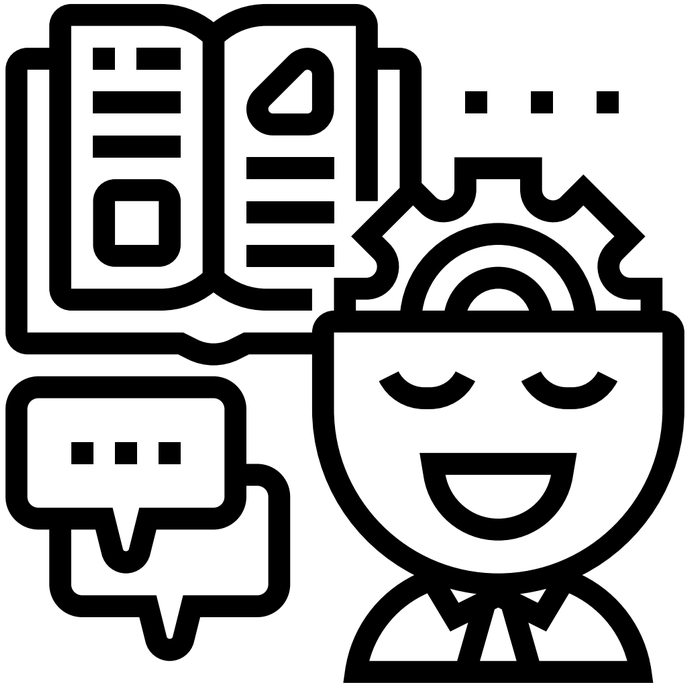
STRENGTHS, TRADE-OFFS, AND NEXT STEPS

FUTURE PROSPECT

It is easy to understand and implement as a feature on the common projects.

CONCLUSIONS

Completed a comprehensive RAG project in .NET, leveraging OpenAI LLMs for development.



THANK YOU FOR YOUR ATTENTION

Q & A